

자바프로그래밍

AI 활용 보고서

XOR 암호화 채팅 프로그램 구현 과정의 AI 프롬프팅 기록

1. AI 활용 개요

주요 활용 AI 도구	OpenAI Codex (GPT-4o 기반)
총 프롬프트 횟수	약 8 회
주요 활용 목적	■ 클래스 설계 ■ 코드 생성 ■ 오류 수정 ■ 개념 질의 □ 테스트 작성 □ 기타
AI 활용 전략 요약	초기 코드는 AI를 활용해 전체 클래스 구조의 큰 틀을 잡고, 이후 받은 프로토타입 코드를 바탕으로 강의에서 배운 최신 자바 문법과 OOP 개념을 직접 적용하며 수정해 나갔다. 추가로 필요한 내용이나 모르는 부분이 생기면 AI에 질의하거나 프로토타입 코드를 요청해 학습한 뒤 직접 코드에 반영하는 방식으로 진행하였다.

2. 프롬프트 이터레이션 로그

※ 최소 5 회 이상의 프롬프트 이터레이션을 기록하세요. 단순 복사·붙여넣기 외에 수정 이유가 반드시 포함되어야 합니다.

▶ □□□□ #1 목적: 전체 클래스 구조 및 초기 코드 생성	
프롬프트 입력	AI 응답 요약
각 클래스별 코드 어떤식으로 구성했는지랑 왜 그런식으로 구성하였는지 전체적으로 설명해줘	Config, XorCipher, PhoneBook, PeerServer, PeerClient, MessageHandler, ChatApp, main 8 개 클래스로 구조를 구성하고 각 클래스의 역할과 설계 이유를 설명하였다. Optional, 람다, 함수형 인터페이스, 스트림 API, 소켓, 스레드가 자연스럽게 포함되도록 구성되었고, XorCipher 의 키 생성 부분은 직접 수정하도록 TODO 로 남겨두었다.
수정 이유 / 다음 프롬프트로의 연결	
전체 구조는 참고하되, XOR 키 생성 알고리즘은 직접 구성하기로 했다. 수업에서 배운 파일 입출력을 활용해 시드값을 파일로 관리하는 방식을 떠올려 다음 질의로 이어졌다.	
▶ □□□□ #2 목적: 암호화 키 시드값을 파일로 관리하는 방식 검토	
프롬프트 입력	AI 응답 요약
금일 자바 과제에서 file input, output 관리 하기를 배웠는데 그러면 내가 만들 키값에 들어갈 시드값들을 모아둔 파일 하나 만들어서 연동하는 식으로 하면 될까? 아님 다른 좋은 방법들이 있을까?	seed.txt 파일에 시드값을 저장하고 XorCipher 가 읽어서 키를 생성하는 방식을 추천하였다. 양쪽 컴퓨터가 같은 파일을 가져야 복호화가 가능하므로 시간이나 랜덤값처럼 양쪽에서 달라질 수 있는 값은 피하도록 안내하였다.
수정 이유 / 다음 프롬프트로의 연결	
파일 시드 방식은 채택하였다. 다만 시드값이 민감한 정보인 만큼 보안 측면도 고려해야 한다고 판단하여, java.security 관련 내용을 추가로 질의하였다.	

▶ □□□□ #3 목적: Config 싱글톤 패턴 적용 및 PeerServer 정적 팩토리 메서드 변환 -

프롬프트 입력	AI 응답 요약
Config 클래스를 싱글톤으로 바꾸고 싶은데 어떻게 하면 돼? 그리고 PeerServer 를 정적 팩토리 메서드 패턴으로 바꾸고 싶은데 프로토타입 코드 줄 수 있어?	Config 는 생성자를 private 으로 막고 static 인스턴스를 클래스 내부에서 생성하는 싱글톤 구조를 제안하였다. PeerServer 는 생성자를 private 으로 막고 defaultPeerServer()·changePortPeerServer() 두 가지 정적 팩토리 메서드를 제공하는 프로토타입 코드를 제시하였다.

수정 이유 / 다음 프롬프트로의 연결

AI 가 제안한 프로토타입을 참고하여 Config 는 싱글톤으로, PeerServer 는 정적 팩토리 메서드 패턴으로 직접 작성하였다. 이 과정에서 정적 팩토리 메서드를 생성자 문법으로 잘못 작성하는 오류가 발생하였고, 이를 AI 에 질의해 문법 구조를 이해한 뒤 수정하였다.

▶ □□□□ #4 목적: MessageHandler 가상 스레드 및 ReceiveTask 내부 클래스 구조로 수정

프롬프트 입력	AI 응답 요약
강의에서 가상 스레드 배웠는데 MessageHandler 에 적용하고 싶어. 그리고 수신 로직을 내부 클래스로 분리하고 싶은데 어떤 구조가 좋을지 프로토타입 코드 줄 수 있어?	Executors.newVirtualThreadPerTaskExecutor() 를 사용해 가상 스레드 ExecutorService 를 만들고 ReceiveTask 를 Runnable 을 구현하는 내부 클래스로 분리하는 구조를 제안하였다. try-finally 로 소켓과 executor 를 반드시 정리하는 프로토타입 코드를 제시하였다.

수정 이유 / 다음 프롬프트로의 연결

AI 프로토타입을 참고하여 기존 일반 Thread 기반 startReceiveThread() 를 가상 스레드와 ReceiveTask 내부 클래스 구조로 직접 수정하였다. 기존 단순 lambda 방식보다 역할이 명확하게 분리된 구조가 강의에서 배운 OOP 원칙에도 더 잘 맞는다고 판단하였다.

▶ □□□□ #5 목적: MessageDTO 불변 객체 도입 및 콘솔 UI 분리 구조 요청

프롬프트 입력	AI 응답 요약
메시지를 그냥 문자열로 보내지 말고 닉네임이랑 시간이랑 내용을 묶어서 DTO 로 만들고 싶어. 그리고 콘솔 출력 부분도 따로 클래스로 분리하고 싶은데 프로토타입 코드 줄 수 있어?	MessageDTO.Message 를 sentTime, nickname, content 세 필드를 가지는 불변 객체로 구성하고 toPacket()·fromPacket()으로 직렬화·역직렬화하는 구조를 제안하였다. ConsoleUI 는 싱글톤으로 구성하고 showMessage()·showConnected() 등 출력 역할을 전담하는 프로토타입 코드를 제시하였다.

수정 이유 / 다음 프롬프트로의 연결

AI 프로토타입을 바탕으로 MessageDTO.Message 를 직접 작성하고, 싱글톤 MessageDTO 저장소로 채팅 기록을 관리하는 구조를 추가하였다. ConsoleUI 도 프로토타입을 참고해 직접 완성하였고, /history·/quit 등 명령어 처리 로직은 handleCommand() 메서드로 직접 분리하였다.

※ 5 회 초과 시 표를 복사해 추가하세요.

3. 최종 코드 선택 근거

※ AI 가 생성한 여러 코드 중 최종 제출 코드를 선택한 이유를 설계 원칙 기준으로 서술하세요.

클래스 / 메서드	적용 설계 원칙	선택 근거 (구체적으로)
CipherSeed	Optional 반환 + 단일 책임	AI 가 생성한 connect() 메서드는 Socket 연결 성공 시 Optional.of(), 실패 시 Optional.empty()를 반환하는 구조였다. null 처리 없이 호출부에서 안전하게 결과를 처리할 수 있고 역할도 명확해 설계 원칙에 부합한다고 판단하여 해당 구조를 그대로 채택하였다.
createKey() / createInitialSeed()	유틸리티 메서드 분리	AI 가 생성한 toHex()·fromHex()는 바이트 배열을 16 진수 문자열로 변환하는 유틸리티 메서드로, 소켓 텍스트 전송에 필수적인 역할을 한다. 로직이 단순하고 목적이 명확하여 수정 없이 그대로 채택하였다.
MessageHandler	단일 책임 + 흐름 분리	AI 가 생성한 serverMode()·clientMode() 분기 구조는 서버와 클라이언트 역할을 명확히 나누고 있어 OOP 설계 원칙에 부합하였다. 내부 세부 구현은 수정하였지만 메서드로 분기하는 전체 흐름 구조는 그대로 채택하였다.
XorCipher	스트림 기반 소켓 I/O	AI 가 생성한 소켓 입출력 구조(BufferedReader + InputStreamReader + PrintWriter, UTF-8)는 한글 깨짐 없이 안정적으로 동작하였다. 스트림 구성 방식이 강의에서 배운 I/O 개념과 일치하고 실제로도 잘 동작하여 그대로 채택하였다.
Config / UserProfile / MessageDTO	스트림 API + Optional 반환	AI 가 PhoneBook 내에 구현한 findMyLanIp()의 스트림 API 구조(filter·flatMap·findFirst)는 네트워크 인터페이스 탐색 로직을 간결하게 표현하였다. 클래스명과 IP 변환 기능을 제거하고 LanNetwork 로 재설계하면서 이 메서드의 스트림 구조는 그대로 채택하였다.

AI 가 생성한 코드에서 직접 수정한 부분

※ AI 코드를 그대로 사용하지 않고 본인이 수정한 부분을 구체적으로 기술하세요.

No	수정한 위치	수정 내용 요약	수정 이유
1	Config 클래스	정적 필드 클래스 → 싱글톤 패턴으로 변경	AI 프로토타입을 참고해 Config 생성자를 private 으로 막고 getInstance()로만 접근하도록 직접 작성하였다. 기존 public

			static 필드 구조는 어디서든 값을 바꿀 수 있어 부작용이 생길 위험이 있었기 때문이다.
2	PeerServer 클래스	생성자 public → private, 정적 팩토리 메서드 추가	AI 프로토타입을 참고해 defaultPeerServer().changePortPeerServer() 팩토리 메서드를 직접 작성하였다. 생성자를 private 으로 막아 객체 생성 의도를 이름으로 명확히 표현하도록 하였다.
3	MessageHandler	Thread → 가상 스레드, ReceiveTask 내부 클래스 분리	강의에서 배운 가상 스레드를 직접 적용하였다. AI 프로토타입을 참고해 수신 로직을 ReceiveTask implements Runnable 내부 클래스로 분리하고 try-finally 로 자원을 정리하도록 직접 작성하였다.
4	XorCipher 키 생성	고정 문자열 키 → 파일 시드 기반 LCG 키 생성으로 교체	AI 프로토타입 구조를 참고해 CipherSeed 를 직접 설계하고 XorKeyMaker 함수형 인터페이스로 연결하였다. 키가 코드에 노출되는 문제를 해결하고 싶었고, 암호화 알고리즘과 상수는 직접 결정하였다.
5	PhoneBook → LanNetwork	IP 전화번호 변환 제거, LAN IP 유효성 검증으로 재설계	AI 가 구현한 PhoneBook 의 IP 변환 공식에서 세 번째 옥텟이 버려지는 문제를 직접 발견하였다. 변환 방식을 포기하고 IP 를 그대로 입력받아 4 개 옥텟을 단계적으로 검증하는 LanNetwork 를 처음부터 직접 설계하였다.

4. AI 한계 및 직접 문제 해결

※ AI 가 틀린 답을 내거나, AI 없이 직접 해결한 상황을 기록하세요.

No	AI 가 실패한 상황	AI 의 오답 / 한계	본인의 해결 방법
1	AI 가 제안한 PhoneBook IP 변환 공식의 구조적 결함	ipToPhoneNumber() 에서 세 번째 옥텟을 버리는 구조라 phoneNumberToIp() 로 원래 IP 를 복원할 수 없었다. AI 가 제시한 공식 자체에 논리적 결함이 있었다.	PhoneBook 방식 자체를 포기하기로 직접 판단하였다. IP 를 그대로 입력받고 4 개 옥텟의 숫자 유효성과 192.168 범위를 단계적으로 검증하는 LanNetwork 클래스를 직접 설계하고 구현하였다.
2	nonce 기반 암호화 강화 방식 제안	AI 가 메시지마다 SecureRandom 으로 nonce 를 생성하고 기본 시드와 XOR 해 키 스트림이 매번 달라지도록 하는 방식을 제안하였다.	테스트 목적의 프로젝트이므로 구조를 단순하게 유지하는 것이 더 적합하다고 직접 판단하였다. 날짜 시드 + LCG + XOR 방식으로 확정하고, nonce 방식은 채택하지 않았다.

		그러나 nonce 를 암호문과 함께 전송하려면 패킷 형식 변경이 필요하여 구조가 복잡해졌다.	
3	java.security.SecureRandom 기반 seed.dat 방식 제안	AI 가 시드를 SecureRandom 으로 생성하고 seed.dat 파일에 저장한 뒤 양쪽이 공유하는 방식을 제안하였다. 그러나 seed.dat 는 결국 수동으로 양쪽에 복사해야 한다는 점에서 cipher_seed.txt 와 다를 것이 없었다.	언어 출시일이라는 의미 있는 날짜값을 시드로 활용하는 것이 더 설명하기 쉽고 프로젝트 의도에도 맞다고 직접 판단하였다. SecureRandom 방식 대신 직접 설계한 날짜 기반 시드 방식을 채택하였다.
4	정적 팩토리 메서드를 생성자 문법으로 잘못 작성	문법 오류 3 가지 (static 생성자 불가, 반환값 없음, new 없이 직접 호출 불가) 는 AI 가 설명하였으나, Config.PORT 를 인자로 받을지 내부에서 참조할지는 AI 가 판단해주지 않았다.	Config 에 PORT 값이 이미 있으므로 생성자 인자로 받으면 중복이 된다고 직접 판단하였다. Config.getInstance().getPort() 를 내부에서 참조하는 방식으로 직접 결정하고 코드를 완성하였다.

AI 의존도에 대한 자기 평가 (200 자 이상)

초기 코드 전체의 큰 틀을 AI 가 만들어준 만큼 처음에는 코드를 이해하는 것 자체가 어려웠고, 정적 팩토리 메서드를 생성자 문법으로 잘못 작성하는 것처럼 개념을 완전히 이해하지 못한 채 AI 코드를 쓰려다 생긴 실수도 있었다. 반면 nonce 방식이나 SecureRandom 기반 시드처럼 AI 가 제안한 방식을 그대로 따르지 않고 프로젝트 목적에 맞게 스스로 판단해 기각한 경험은 AI 를 단순히 따르는 것이 아니라 도구로 활용하는 방법을 배우게 해줬다. AI 가 만들어준 프로토타입을 참고해 가상 스레드 적용이나 DTO 설계처럼 강의에서 배운 내용을 직접 코드에 녹여보는 과정이 가장 도움이 됐고, 앞으로도 설계 판단은 직접 내리고 구현의 세부적인 부분에서만 AI 를 보조 도구로 활용하는 방식을 유지할 계획이다.

※ AI 가 생성한 코드와 본인이 직접 작성·수정한 코드의 비율을 추정하고, AI 활용의 장단점을 기술하세요.

구분	AI 생성 코드 비율	직접 작성·수정 비율
추정 비율	약 65%	약 35%

AI 활용 소감 (장점·한계·배운 점, 300 자 이상)

AI를 활용하면서 가장 좋았던 점은 초기 클래스 구조를 빠르게 잡고 필요한 개념의 프로토타입 코드를 즉시 받아볼 수 있었다는 것이다. 가상 스레드나 `ReceiveTask` 내부 클래스처럼 강의에서 배운 개념을 적용하고 싶을 때 프로토타입을 요청해 참고하면서 직접 수정하는 방식이 학습에도 도움이 됐다. 반면 AI 코드를 그대로 두면 구조적 결함을 나중에야 발견하는 경우가 있었다. `PhoneBook`의 IP 변환 공식처럼 실제로 돌려보기 전까지 문제를 파악하기 어려웠고, `nonce`나 `SecureRandom`처럼 AI가 제안한 방식이 무조건 좋은 것은 아니라는 것도 알게 됐다. 이번 프로젝트를 통해 AI는 직접 방향을 정한 뒤 그 방향에 맞는 구현 방법을 찾을 때 쓰는 것이 가장 효과적이라는 것을 배웠다.